Understanding Performance Bottlenecks of Large-Scale Agent-Based Simulations

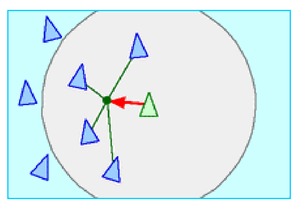
Amina Sokoli A. Giray Yağlıkçı
CISPA Helmholtz Center for Information Security,

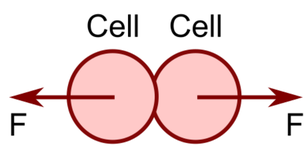
Motivation

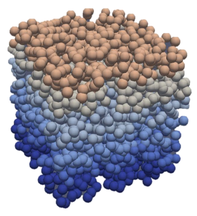
Agent-based simulations are a powerful simulation tool used to model complex real-world phenomena in diverse scientific fields. Their hardware performance characteristics remain poorly understood. As simulations scale to multi-billions of agents, data movement between compute units and memory becomes a critical bottleneck. **No prior work rigorously investigates these bottlenecks**, leaving a gap in our understanding of what limits scalability.

Introduction

- Agent-based simulations **model** real-world as populations of **autonomous agents** interacting in an environment. Their memory access patterns *resemble graph workloads'* pointer-chasing traversal, but with dynamic, spatially-evolving neighbor structure.
- We evaluate performance bottlenecks in agent-based simulations by profiling **BioDynaMo**'s epidemiology use case on **Google Cloud C4 VMs** across agent counts from **10 million to 1 billion**.







Methodology

Table 1: Google Cloud C4 VM Specs

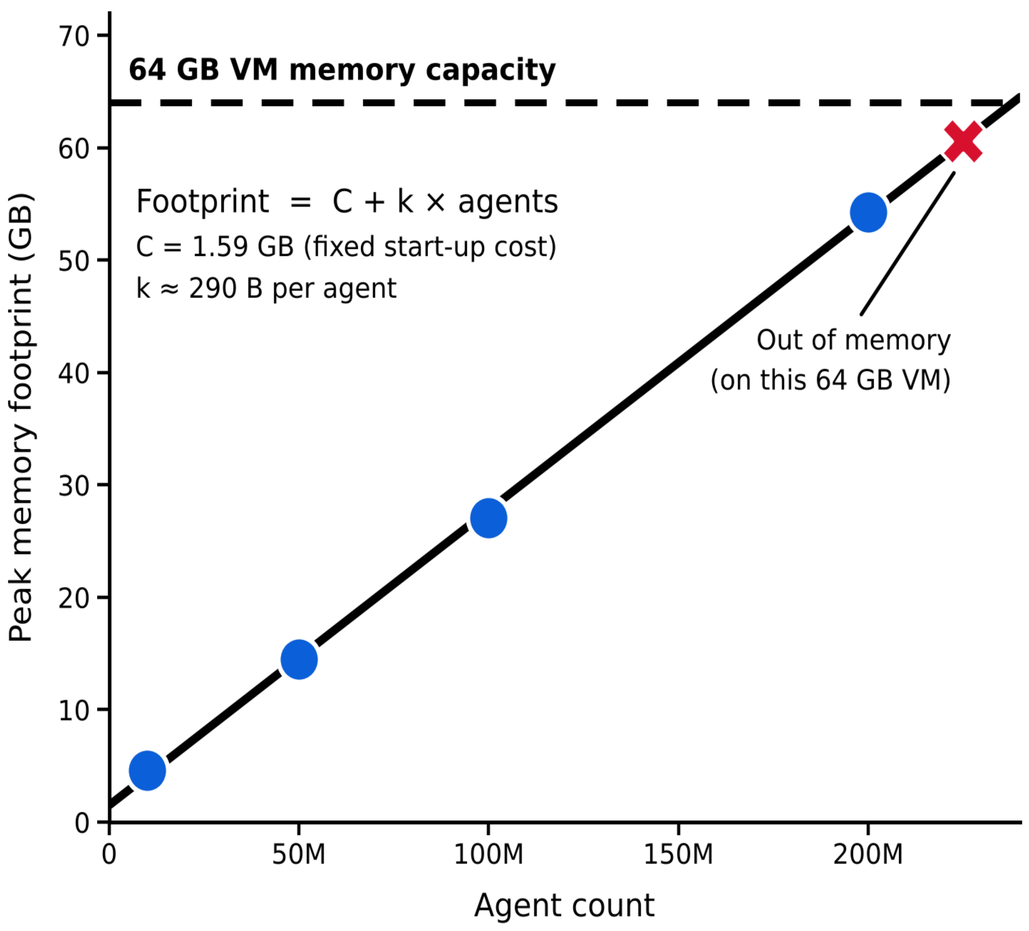
Processor	16-core Intel Emerald Rapids
Memory	64GB with four NUMA domains
OS	Ubuntu 22.04 LTS

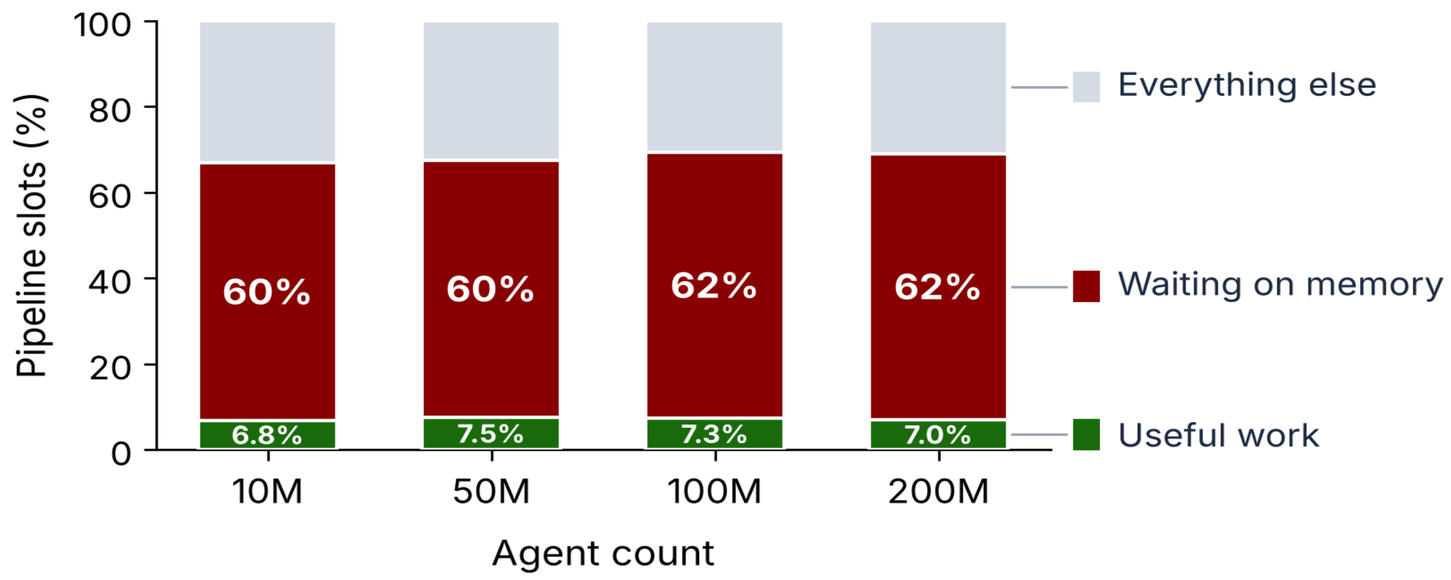
- 3 runs per agent count x 3 individual VMs
- PMU counters collected via `perf` sampled at **1 Hz**
- **16, 8, 4, 2 and 1 threads** executions

Table 2: PMU Statistics

PMU events	Description	Metric
Instructions	Number of instructions	IPC
Cycles	Number of CPU cycles	IPC
MEM_LOAD_RETIRED.L1_MISS	Number of retired instructions that cause an L1 miss	L2/L1 miss ratio
MEM_LOAD_RETIRED.L2_MISS	Number of retired instructions that cause an L2 miss	L2 MPKI, L2/L1 miss ratio
TOPDOWN.MEMORY_BOUND_SLOTS	Pipeline slots that are memory bound	Mem-bound fraction
TOPDOWN.SLOTS_P	Total number of slots in the processor pipeline	Mem-bound fraction

Results and Analysis





Low compute utilization + Mem-stalls

Memory footprint scales linearly with agent count and bounds simulation complexity

IPC remains consistently low across all runs with a value of ~0.4 on a 16-core system, dropping from ~0.65 at 1 thread

It's the Memory, Stupid!

- Short computation wrapped around irregular walks over a neighbor structure
- Most of the execution time goes to waiting for memory
- Limited by the memory capacity

CISPA HELMHOLTZ CENTER FOR INFORMATION SECURITY

S4L

Future works:

- Test on **bare metal** machines to rule out virtualization;
- **Characterize temporal and spatial locality** for a precise DAMOV classification;
- **Extend** our experiments to **architectures designed for highly-parallel execution** (e.g., GPUs, PuD)